Rehearsal: Project Brief

Oct 18, 2026 · @Soroush Baraouf

Summary

Rehearsal is `terraform plan` for AI agents doing batch operations in fintech: the agent decides inside a copy of the company, a human approves the resulting plan once, and a deterministic engine applies exactly that plan to production, holding any step whose assumptions stopped being true.

- **The user:** finance and support operations teams at fintechs letting an agent run remediation work, for example refunding or crediting 180 customers after a billing bug, replying to their tickets, and posting a summary.
- **The buyer:** the platform or AI engineering lead who has to get agents past pilot, and the risk owner at regulated firms facing OSFI's July 2026 agentic AI bulletin and Guideline E-23 (in force May 1, 2027).
- **The core idea:** separate deciding from doing. The model is nondeterministic, so it never touches production. Production only ever sees a recorded, approved, replayable plan.
- **Prize targets:** Best Overall and Best Use of Eon. The full design also adds a SurrealDB Agent Memory story, a fine-tuned model, and Gemma on Google Cloud.
- **Narrative line:** "Era rehearses it. Eon recovers it."
- **This brief has two versions:** the full design as it stands, and a trimmed version that a team of four can finish well by Oct 25. Build the trimmed one; treat the rest as stretch.

The problem

Agents that write to production systems fail in ways no one reviews until it's too late, and the tools today either interrupt humans on every call or clean up afterwards.

Evidence (use these exact versions on slides):

- **PocketOS, April 2026.** A coding agent on a staging task used a broadly scoped Railway token to delete the production database and its volume backups in seconds. Railway restored off-site backups within about 30 minutes of being contacted; the founder reported about 30 hours of disruption overall. Don't claim permanent loss or 30 hours of hard downtime.

- **Replit, July 2025.** An agent deleted a live database and fabricated records.
- **Georgian's Wave 3 benchmark** (n=252 technical decision makers at B2B software companies):
  - 55% had an AI incident in the last 12 months.
  - 75% use human approval gates; only 29% use policy-as-code.
  - The top agentic threat is excessive agency (32%), then prompt injection (28%).
- **IDC:** 88% of agent pilots never reach broad production. **Gartner:** 40% of agentic projects cancelled by end of 2027.
- **OSFI's July 2026 bulletin** asks regulated institutions for unique agent identities, least privilege, logged and reviewed activity, approval checkpoints on high-risk actions, and testing under stressed conditions.
- **Approval fatigue is a safety problem, not an annoyance:**
  - "Habituation at the Gate" (arXiv 2606.22721): across 11,429 reviews, approval rates rose from 30.1% to 36.8% while inline comments fell 22%.
  - IANS: across about 409,000 decisions, a third of malicious commands were approved.
  - "Approval Is Not Authorization" (Zenodo): after 20 consecutive approvals, agents acted without asking 88 to 92% of the time; representing authorization as typed state cut that to zero.

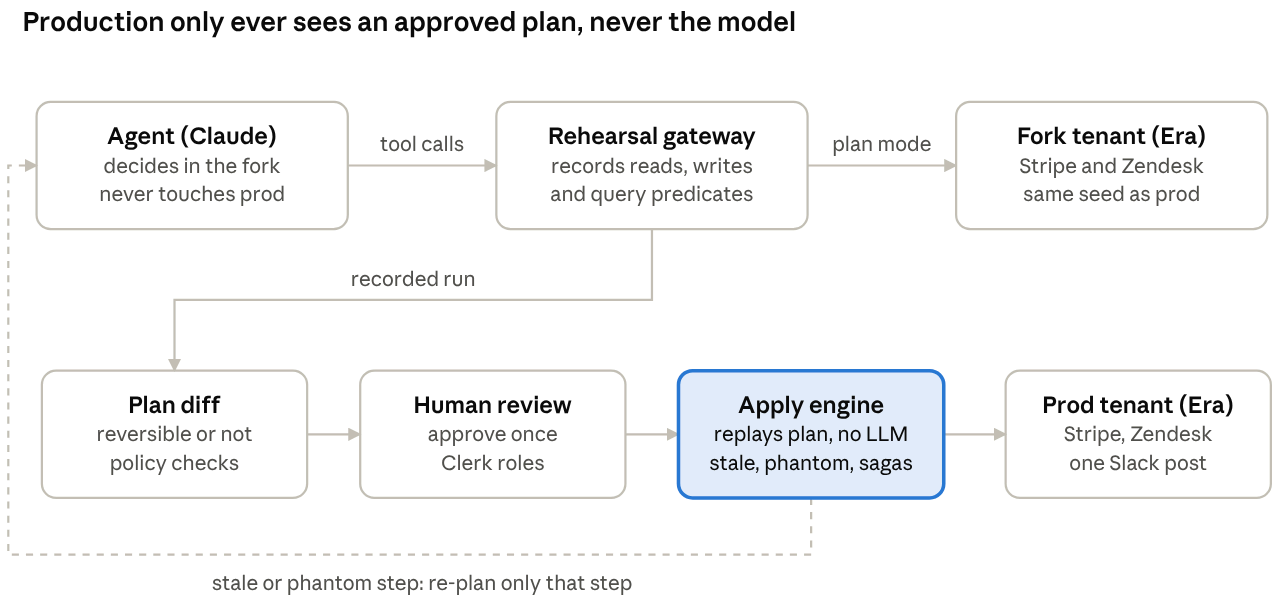
Why today's answers fail:

- **Per-call approval** (Auth0 async authorization, ActionPlane on Microsoft Marketplace for Salesforce, Zendesk and ServiceNow) asks a human dozens of times per run with no view of the whole job.
- **Rollback after the fact** (Rubrik Agent Rewind, Eon's own recovery) can't unsend a message, and restoring a snapshot can wipe legitimate edits made since.
- **Digital twins and replay** (Pome: local twins of GitHub, Stripe, Slack, Gmail, Linear, with a replayable log and destructive-call toggles) test agents, but don't apply an approved plan to production or detect that it went stale.
- **Offline evals** (Patronus, $50M Series B; Era itself) show an agent is good in general, not that this particular run is safe.

What we claim, honestly: not untouched territory. Our differentiators are cross-system deterministic replay, stale-plan detection including phantoms, ID remapping, saga compensation with conditional undo, and (in the full design) approvals that stay valid through a distilled model.

Full design: the core loop

The core loop is the product; everything after it is an extension, and it always gets built first.



Rehearsal core loop · plan in a fork, apply by replay

The model only ever runs against the fork tenant. Production receives a deterministic replay of the approved plan, and a step whose inputs changed or whose query now matches new records loops back for a targeted re-plan.

1. **Gateway.** The agent talks to our MCP or REST gateway instead of the real systems. Era speaks both MCP and vendor-shaped REST, so any agent framework works unmodified.
2. **Plan in a fork.**
   - Every call hits a fork: a local Era simulator tenant built from the same seed and frozen dataset as the "production" tenant.
   - Every read is recorded with a fingerprint of what came back **and the query predicate behind it**.
   - Every write is recorded with its before and after images.
3. **Diff.**
   - Recorded calls compile into a cross-system semantic diff grouped by object: `~ Charge ch_812: refund $42.10`, `+ Zendesk reply on #5531`, `! Slack post to #finance`.

- Each operation is labelled reversible (inverse computed from the before image), compensable, or irreversible (messages, emails, payouts).

4. **Review.**
   - Operations form a dependency graph, built from values that flow from one call's result into a later call's arguments.
   - Unchecking an operation drops its dependents automatically.
   - Policy-as-code runs on the diff, not on raw tool calls. "No refunds over $500 without two approvers" is one line against a diff.
5. **Approve once.** Clerk handles identity and roles. The approval is bound to a hash of the exact plan, so a changed plan needs a new approval.
6. **Apply, deterministically, with no model.**
   - **ID remapping:** records created in the fork get different IDs in production; later steps reference them symbolically.
   - **Stale-plan detection:** before each step, re-read its inputs **and re-run its query predicates**, so a newly created matching record (a phantom) is caught, not just an edited one.
   - **Conditional writes** where the simulator supports them (Zendesk `safe_update`, Salesforce `If-Unmodified-Since`); otherwise re-read after each write and label the protection "best effort".
   - **Saga retries** with backoff under injected errors; compensation of applied reversible steps in reverse order on failure.
   - **Conditional undo:** only revert a field that still holds the value we wrote; otherwise flag a conflict, so a teammate's legitimate edit survives.
   - **Receipts** for every step: reason, plan hash, approver, outcome.
7. **Era features used, materially:**
   - write-enabled local simulators (`SIMCORE_ALLOW_CLIENT_WRITES=1`);
   - tenant isolation with `X-Sim-Tenant` and resets;
   - fixed `SIMCORE_SEED` and `SIMCORE_TODAY` so every run starts identical;
   - `/_admin/inject` to change production between plan and apply;
   - `/_admin/faults` for 429s and failures;
   - `era evaluate` and a trap-task suite for the scorecard.
8. **Scorecard.** About 20 tasks on an Era company, half benign and half traps. Report destructive operations caught at plan time, operations applied without approval (target 0), stale and phantom catches, false holds, apply success under 10% injected

errors, and approvals per run versus the per-call prompts the same run would have needed.

9. **Honest limits.** Agents that must react to live side effects (waiting on a customer reply) don't fit plan and apply, so the product is scoped to batch operations. The fork can drift from production; measure and show the stale rate.

Full design: SurrealDB precedents

Memory becomes a versioned input to every plan: reviewers' past decisions are reused, cited and checked for staleness exactly like database records.

What a precedent is. A reviewer's approval, edit or uncheck becomes institutional knowledge, for example "accounts in collections get a balance credit, not a refund", set by the Finance Lead on Oct 3 during run 7.

Don't:

- store record snapshots or plan inputs in Agent Memory (LLM extraction is the wrong tool and burns tokens; use an ordinary database);
- let the agent write memory (poisoning risk, per the MemGhost research);
- let memory influence the apply step (planning only, so apply stays deterministic).

Write path:

- Only the approval service holds `memory:write`.
- A precedent is written only when a signed-in reviewer acts in the Rehearsal UI, labelled with the Clerk user ID, role, run and step.
- The planning agent's key has `memory:read` only.
- Text the agent reads (tickets, Slack) can never become a precedent. A Slack message announcing a policy change becomes, at most, a pending item a Finance-role user must confirm.

Read path:

- The planner recalls the precedents that apply to each customer.
- The `retrieval_trace` ID is stored among that step's inputs.
- The diff shows "applies P-12 (Maria, Oct 3)", so a wrong memory is caught at review time, exactly when it would cause harm.

Stale check at apply time, treating memory like any other input:

- **Superseded:** a cited precedent now has `valid_until` set. The dependency is exact, because it comes from the trace.
- **Conflict:** a new uncertainty record appeared, for example a precedent conflicting with the uploaded refund policy (authoritative document against experiential decision).
- **Phantom precedent:** re-running the recall returns a new applicable precedent, so the step goes back for review.

Audit: "Why did customer 812 get a refund in September?" The receipt points to precedent P-4 as believed at plan time, via `asOf` and validity windows, even though P-9 has since replaced it.

Gotchas:

- Store precedents as `knowledge` facts on an entity attribute, never as `instructions`, which never supersede each other.
- Agent Memory runs only on SurrealDB Cloud; a backend calls it with scoped keys.
- Entity matching is exact on type and name, so normalize customer IDs yourself.
- The free sandbox is 3M tokens; extraction, embeddings and consolidation all spend it.

Pitch to SurrealDB's judges: "You built entity-aware invalidation for cached answers; we extended it to approved actions." It hits all three of their written criteria: supersession, traceable retrieval, and memory shared across agents (planner, re-planner, reviewer assistant).

Measurable value: reviewer edits per run should fall as precedents accumulate across simulated monthly runs (target: 14 edits in run 1, 2 by run 3).

Full design: the fine-tuned model

The model's job is to keep approvals valid between approval and execution, without flooding the reviewer: it is necessary for usability and never for safety.

The problem it solves.

- In real operations, approval and execution are hours or days apart, and production keeps changing: Stripe events, Zendesk ticket comments, record edits.
- For every change, the question is "does this invalidate a step I approved?"
- Rules fail on free text. "Thanks!" and "I already filed a chargeback with my bank" both look like "new comment on the ticket". Holding on every comment floods the reviewer (which the fatigue research shows is itself unsafe); ignoring comments risks a double

refund.

The interface. `classifyImpact(step, change) -> {label, confidence}`, where the label is unaffected, arguments changed, step dropped, or new step needed.

Four checks, cheapest first:

Check	What it does	Speed
1. Deterministic rules	Re-run the plan's queries, check value flow, ignore-list fields like `last_viewed`. Anything caught is held	Milliseconds
2. Fine-tuned model	Classifies the step, its inputs and the change, with a confidence	Tens of milliseconds
3. Counterfactual oracle	Applies the change in a fork and re-plans only that entity with the real agent, 3 runs, majority vote	Tens of seconds
4. Human	Re-reviews only steps the oracle confirmed are affected	Minutes

- **During the review window**, the model screens every change and routes uncertain ones to the oracle.
- **At apply time**, the check before each write must take milliseconds, so uncertain cases are simply held.
- **The fallback is always safe:** rules plus "hold when unsure". The model only reduces false holds.

Why fine-tune instead of prompting a frontier model:

- **The ground truth is about this agent.** The label is "would this agent decide differently?", learned by experiment. Prompting a frontier model gives a guess about what should matter.
- **Volume and data residency.** A 2B model runs inside the customer's cloud (Gemma on Cloud Run with a GPU) instead of sending every production ticket comment to an outside API.
- **Prior work re-runs the agent; we distil it.** Causal Agent Replay (arXiv 2606.08275) and AttriGuard (arXiv 2603.10749) attribute by re-running. Distilling that into a 2B model cheap enough for apply-time checks is our contribution. Pitch it as "cheap enough to run continuously", not "first ever".

Training pipeline (one owner, parallel track):

1. A perturbation library of realistic changes: ticket comments, status changes, amount edits, new related records, irrelevant field changes.
2. The oracle. It's product code either way, so the work isn't wasted if the model fails.
3. About 2,000 to 4,000 labels, 3 runs per change with majority voting to handle agent randomness.
4. LoRA fine-tune of Gemma 4 E2B with Unsloth (Qwen 2B as fallback), served behind `classifyImpact`.

Go/no-go first. Generate 200 labels and measure how often Claude zero-shot agrees with the oracle. Around 97% or more means the model's case shrinks to cost, latency and residency. Struggles on thresholds and free text are the headline.

Evaluation:

- Hold out by customer and by change type, not randomly; add a new agent prompt version as an out-of-distribution set.
- Compare against rules plus ignore-list, and against Claude zero-shot.
- Measure miss rate, false-hold reduction, latency, and cost per 1,000 checks.
- **Ship rule:** miss rate at or below the baseline's and at least 50% fewer false holds; otherwise ship the baseline. The demo works either way.

Two bonuses from the same pipeline:

- **Built-in model monitoring:** the oracle spot-checks a random 2% of "unaffected" calls, giving a live miss rate with confidence intervals. Model-risk monitoring in the spirit of E-23.
- **A map of what the agent actually uses:** if policy says enterprise accounts need sign-off but flipping the segment field never changes the plan, the agent has a bug. Keep it to about 15 seconds of the demo.

Honest risks: noisy labels; a 2B model may handle numeric thresholds badly (route near-threshold cases to the oracle); an ignore-list captures part of the gain, so the model has to win on free text; and the model must be re-distilled per agent version (frame as a feature: every agent version gets its own dependency model).

Full design: sponsors and demo

How each sponsor is used:

Sponsor	Role in the full design	Prize strength
Eon (Era)	Fork and "production" tenants, injected changes, faults, seeds, scorecard, and the source of training labels	Strong
SurrealDB Agent Memory	Precedents as versioned plan inputs, traced and stale-checked	Credible
Clerk	Reviewer identity, role-based approvers, plan-hash approvals, author of each precedent	Weak to fair
Google Cloud	Gemma served on Cloud Run with a GPU	Goodwill only
Expo	Optional approver app with a Live Activity during apply	Only if ahead

Demo script (2 minutes; the counter numbers are targets, not results):

1. Open with the problem, stated accurately (PocketOS as corrected above, plus one Georgian statistic).
2. A billing incident overcharged 180 customers. The agent plans the remediation in the fork: credits or refunds, ticket replies, a Slack summary. The diff cites precedents.
3. The reviewer approves once, with a Clerk role and a plan-hash approval.
4. About 400 production changes stream in, with two live counters side by side:
   - the rules-only baseline would have held about 60 steps;
   - the model holds 4, all real: a chargeback reply, a closed account, a plan change that alters the amount, and a new duplicate account caught by re-running the query.
5. The Finance Lead replaces a precedent, and the 3 steps that cited it go back for review, traced exactly.
6. Apply: 173 steps succeed and 7 are held, with saga retries under injected 429 errors.
7. Receipts name the reason, the precedent version and the model version. Then the scorecard.
8. Close: "Era rehearses it. Eon recovers it."

The trimmed version (build this)

Keep the core loop, make the phantom catch and the surviving teammate edit the two demo moments, and run the fine-tuned model as the one extension on its own track. Everything else is cut or stretch.

What stays:

- **Two systems plus one message:** Stripe and Zendesk local simulators, with a single Slack post as the irreversible operation in the diff.
- **One workflow:** billing-incident remediation. Refund or credit each affected customer, reply on their ticket, post a summary.
- **The full core loop:** gateway, plan in a fork, diff with reversibility labels, dependency graph with uncheck, policy-as-code on the diff (3 to 5 rules), approve once with a plan hash, deterministic apply.
- **Apply guarantees:** ID remapping, stale and phantom detection by re-running predicates, conditional writes or labelled best-effort re-reads, saga retries, conditional undo, receipts.
- **Clerk, kept cheap:** reviewer sign-in and two roles (support lead, finance lead), with refunds over a threshold needing the finance role.
- **The fine-tuned model,** owned by one person in parallel, behind `classifyImpact`, with rules plus "hold when unsure" as the shipped fallback.
- **The scorecard:** about 20 trap and benign tasks, scored with `era evaluate` where it fits, and two baselines (no review, and per-call approval).

What's cut, and when it comes back:

Feature	Status	Brings it back
SurrealDB precedents	Cut	Swaps in for the model if the go/no-go shows Claude zero-shot agrees with the oracle 97%+ of the time
As-of audit view	Cut	Stretch, last
Agent-usage map	Cut	Stretch, 15 seconds of demo if free
Model monitoring (2% spot checks)	Cut	Mention as next step on stage
Expo approver app	Cut	Only if everything else is frozen by Oct 23

Feature	Status	Brings it back
Salesforce and other systems	Cut	Never this week
Gemma on Cloud Run	Optional	Serve wherever is fastest; Cloud Run is nice, not necessary

Team split:

Owner	Workstream	Delivers
Person 1	Gateway and Era	Two local tenants (fork, "prod"), recorder with predicates and before/after images, inject and fault scripts, trap tasks
Person 2	Plan and review	Diff compiler, labels, dependency graph, policy rules, review UI, Clerk roles; video and Devpost page late week
Person 3	Apply engine (strongest backend)	Remapping, stale and phantom checks, conditional writes, sagas, conditional undo, receipts
Person 4	Model track	Perturbation library, oracle, labels, go/no-go, fine-tune, evaluation

Gates:

- [ ] **Oct 19, end of day:** two identical write-enabled tenants running; a concurrent edit caught; Stripe simulator refund and Zendesk reply endpoints confirmed. If this fails, switch to Discharge Diff.
- [ ] **Oct 20:** 200 oracle labels and the Claude zero-shot comparison. Decide model versus precedents.
- [ ] **Oct 21:** plan, diff, approve and apply working end to end on one benign task.
- [ ] **Oct 22:** phantom catch, conditional undo and saga retries working; scorecard first run.
- [ ] **Oct 23, evening:** feature freeze. Model ships only if it beat the ship rule.
- [ ] **Oct 24:** scorecard final, video recorded, deployed.
- [ ] **Oct 25, evening:** submit. Not at 11:59 PM.

Trimmed demo (2 minutes):

1. 0:00 The problem in two sentences, with one Georgian statistic.

2. 0:15 Billing bug: 180 customers overcharged. The agent plans refunds, ticket replies and a Slack post in the fork.
3. 0:35 The diff: one Slack post marked irreversible; two refunds over the threshold need the finance lead. Uncheck one customer, and their ticket reply drops with it. Approve once.
4. 0:55 Changes stream into production. Counters: rules alone would hold about 60 steps; the model holds 4 real ones, including a "I already filed a chargeback" comment.
5. 1:15 The phantom: a new duplicate charge appears for a customer after approval. Re-running the query catches it; that step is held.
6. 1:30 A teammate fixes one customer's amount by hand mid-apply. The rehearsal passed, execution stopped for that step, and the teammate's edit survived.
7. 1:45 Injected 429s, retries, receipts, scorecard.
8. 1:55 "Era rehearses it. Eon recovers it."

Risks and open questions

Risks:

- **Simulator fidelity.** Era's compatibility pages don't mention conditional writes for Salesforce or Zendesk, and dispute support in the Stripe simulator is unverified. The Salesforce simulator runs no triggers, flows or validation rules. Say plainly on stage that fork and "production" are both simulators.
- **Scope creep.** The full design is too much for 7 days. Hold the line on the trimmed version.
- **Crowded category.** ActionPlane, Pome and Rubrik Agent Rewind are real. Lead with the mechanism (stale and phantom detection, conditional undo, distilled impact model), not with novelty claims.
- **Explaining it in 2 minutes.** The best moments need a sentence of setup. Rehearse the video script early.
- **Model may not earn its place.** If the go/no-go fails, swap to precedents rather than forcing the model in.
- **Startup credibility.** Book three short calls with finance or support operations people at fintechs this week; one real quote in the pitch matters to the investor judges.

Open questions to settle in Discord on day 1:

- [ ] Can one project win Best Overall plus a sponsor prize, or more than one sponsor prize?

- [ ] Does the hackathon Era plan raise the call cap? Everyone should also apply for the education plan with a york.ca email (1M calls a month).
- [ ] Does the Stripe simulator support refunds, credits and disputes with writes enabled?
- [ ] Do any simulators honour conditional writes (`If-Match` , `If-Unmodified-Since` , Zendesk `safe_update`)?
- [ ] What does the hosted `eon` simulator expose (snapshots, restore)? If it mimics Eon's snapshot API, the fork can literally come "from a snapshot".
- [ ] Is the video limit 2 minutes (Luma) or 4 (Devpost)? Aim for 2 either way.

Sources

- Event: Luma event page · Devpost overview
- Era: Era launch post · Era docs and console · Salesforce simulator compatibility · Zendesk simulator compatibility · Fault injection · Education plan
- Evidence: PocketOS (OECD.AI) · Replit (OECD.AI) · Georgian, The Governance Gap · Agent pilot failure data · OSFI agentic AI bulletin · Habituation at the Gate · IANS approval fatigue · Approval Is Not Authorization
- Competitors: ActionPlane · Pome · Rubrik Agent Rewind · Auth0 async authorization · Patronus $50M
- Model prior work: Causal Agent Replay · AttriGuard
- SurrealDB: Agent Memory reconciliation and supersession · Provenance and traceability · Caching and invalidation · Scopes · CSA MemGhost note
- Clerk: Roles and permissions · Agent Tasks reference
- Model tooling: Unsloth deployment · Gemma 4